

```
[root@legacy alok]# x86info -c
x86info v1.24. Dave Jones 2001-2009
Feedback to <davej@redhat.com>.

Found 2 CPUs
-----
CPU #1
Family: 0 Model: 0 Family: 6 Model: 15 Stepping: 6
CPU Model: Core 2 Duo [82]
Processor name string: Intel(R) Core(TM)2 CPU          T5500 @ 1.66GHz
Type: 0 (Original OEM) Brand: 0 (Unsupported)
Number of cores per physical package=2
Number of logical processors per socket=2
Number of logical processors per core=1
APIC ID: 0x0 Package: 0 Core: 0 SMT ID 0

Cache info
L1 Instruction cache: 32KB, 8-way associative. 64 byte line size.
L1 Data cache: 32KB, 8-way associative. 64 byte line size.
L2 cache: 2MB, 8-way associative. 64 byte line size.

L3 info
Instruction TLB: 4x 4MB page entries, or 8x 2MB pages entries, 4-way associative
Instruction TLB: 4K pages, 4-way associative, 128 entries.
Data TLB: 4MB pages, 4-way associative, 32 entries
L1 Data TLB: 4KB pages, 4-way set associative, 16 entries
L1 Data TLB: 4MB pages, 4-way set associative, 16 entries
Data TLB: 4K pages, 4-way associative, 256 entries.
64 byte prefetching.
-----
CPU #2
Family: 0 Model: 0 Family: 6 Model: 15 Stepping: 6
CPU Model: Core 2 Duo [82]
Processor name string: Intel(R) Core(TM)2 CPU          T5500 @ 1.66GHz
Type: 0 (Original OEM) Brand: 0 (Unsupported)
Number of cores per physical package=2
Number of logical processors per socket=2
Number of logical processors per core=1
```

Figure 4: Cache information using the x86info tool

command—it's a huge one—and look for the part shown in Figure 3.

So my stride would be 32/4, or 8.

Now, let's create the tuned array using the chunk size and stride value. We'll use the `mdadm` command as shown:

```
# mdadm -C /dev/md0 -l5 -n 3 -C 32 /dev/sda7
/dev/sda8 /dev/sda9
```

Now format the device using the stride value. For an ideal filesystem, you can assume a block size of 2 KB (that is, 2048 bytes). But here I am using the actual block size of my filesystem, which is 4 KB or 4096 bytes.

```
# mkfs2fs -b 4096 -E stride=8 -j /dev/md0
```

That's it! Our job is done. Now just mount the tuned RAID array to some directory (`/raid-location`). You can use the `/etc/fstab` file to make it permanent.

Mounting the journal to the external filesystem

Till now the journal has been on the same filesystem that can lead

to performance degrade. Naturally, it's always better to mount the FS journal to some external or separate filesystem.

Step 1 First `umount` the RAID device:

```
# umount /raid-location
```

Step 2 Check the block size of the RAID filesystem using the `dumpe2fs /dev/md0` command.

Step 3 Create a partition using the `fdisk` command that has to be used for the journal in future.

Step 4 Format the new partition that you had just created to use as the journal device:

```
# mkfs2fs -O journal_dev -b 4096 -L raid_journal
/dev/sda10
```

Step 5 Before using the new journal filesystem, remove the existing journal from `/dev/md0`:

```
# tune2fs -O ^has_journal /dev/md0
```

Step 6 Now since the old journal has been removed from `/dev/md0`, the new partition `/dev/sda10` can be used to hold the journal for `/dev/md0`:

```
# tune2fs -j device=/dev/sda11 /dev/md0
```

Step 7 Mount the RAID device now:

```
# mount /raid-location
```

Locality of reference

CPU cache (L1/L2/L3 cache) is a very important factor in performance. Its speed is normally twice the speed of RAM and is used to hasten the memory retrieval process. Without a cache the life of the CPU will be like that of librarians without a storage-shelf behind them. So every time people request a book or need to store it, they have to climb up the ladder to place the book or get the book from the main shelf (main memory) itself. That will obviously slow down the librarian's performance and lower the customer's satisfaction levels.

The cache is divided into a few lines. Each line is used to cache a specific location in the memory. We have separate cache instructions for the processor: I-cache, data instructions, and D-cache. This classification is also known as the Harvard Memory Architecture.

Every cache has an associated cache-controller. So whenever the CPU asks for any reference from the main memory, the cache-controller first checks if that requested reference is in the cache. If the reference is found there, the request is catered to from there itself, without going to the main memory. So this significantly increases performance. This is known as cache-hit.

However, if the request is not found there (known as cache-miss), its requested location is brought from the main memory to the cache for reference (known as cache-line-fill). On a multi-processor system, if one CPU cache gets a cache-line-fill, then the first CPU must inform the second CPU about this. So the other CPU